

Agent specifications an AI coding agent can execute

A spec is a **contract** the building agent reads, fails against, then writes code to pass. After two years of agents going off the rails, the model was almost never the thing letting me down – **the spec was**. Write it once, up front, and the gap between a vague prompt and an executed spec becomes the gap between **reviewing code** and **rewriting it**.

Don't hand-write it. The `/specify` interviewer builds this spec with you – one question at a time – then emits the phase-scoped build prompt (or, for a skill / declarative agent, the artifact itself). → github.com/hmbseaoatter/agent-specification-toolkit

Anatomy of a specification SIX BLOCKS · WRITTEN IN THIS ORDER · ONE FILE PER AGENT, FEATURE, OR SKILL

01 Outcome

What a user can newly do – not what the system does. The anchor everything else aims at.

Make it **measurable**: "reaches dashboard in <2 min, no support." If an agent passes tests but misses this, the spec failed.

02 Scope

What's in – and, just as load-bearing, what's out.

Write the **out-of-scope** list too. With no exclusions an agent builds the most complete version it imagines. Park future work as **V1**

03 Constraints

Your stack, and what the agent must not touch or assume.

Stops it "improving" your architecture mid-task – this block alone **cuts ~80% of rework**. "No new packages without flagging."

04 Prior decisions

What was already settled, and why. Agents are stateless across sessions.

This is the answer to "**why this way?**" before it's asked – your project's institutional memory. Input you author.

05 Requirements **EARS**

Each requirement = one trigger + one verifiable response.

Use the five **EARS** patterns below. The agent reads one and writes its test *before* a line of code. **Trace** each to a check ↓

06 Acceptance criteria

The pass/fail gate the agent runs after building.

Specific & **machine-checkable**: "email arrives within 5s," not "email works." Don't mark done until all pass – by test, not by vibe.

The one word that does the work: **SHALL**

SIGNAL WORD

"WHEN... **SHALL**..." sounds formal, but it does one thing: it turns a preference into a contract a test can check. The keyword tier is load-bearing – borrow it from **RFC 2119 / 8174** so readers know the words mean something.

SHALL = MUST

Absolute, verifiable requirement. Identical meaning – pick one and stay consistent.

SHOULD

Recommended; deviation allowed with reason.
Not a requirement.

MAY

Truly optional. Pure discretion, no obligation either way.

Why this guide only ever uses SHALL: in a requirements block, everything is mandatory by design. If you catch yourself writing "should," that line isn't a requirement – **cut it, or move it to Prior Decisions**.

The five EARS patterns EASY APPROACH TO REQUIREMENTS SYNTAX · EX-ROLLS-ROYCE AEROSPACE

UBIQUITOUS · ALWAYS ACTIVE

The system **SHALL** [response].

EVENT-DRIVEN · TRIGGERED BY ACTION

WHEN [trigger], the system **SHALL** [response].

STATE-DRIVEN · WHILE A CONDITION HOLDS

WHILE [state], the system **SHALL** [response].

UNWANTED · ERROR HANDLING

IF [bad condition], the system **SHALL** [response].

OPTIONAL FEATURE · BEHIND A FLAG – THE TWO PATTERNS THE ORIGINAL TEMPLATE FORGOT ARE WHILE & WHERE

WHERE [feature is enabled], the system **SHALL** [response].

Worked example — password reset REQUIREMENT → ITS MATCHING CHECK, TRACED

REQUIREMENTS (EARS)

WHEN a user submits a registered email, the system **SHALL** send a reset link within **5s**.

IF a token has expired, the system **SHALL** show "link expired" + redirect to /forgot.

→
TRACES
TO

ACCEPTANCE CRITERIA

[✓] valid email → reset email arrives within 5s

[✓] expired link → error shown, redirect to /forgot

• **Type & value discipline** for deterministic code: static typing so a **float can't silently become an int**, plus **immutability** for constants – a passing type-check is an acceptance criterion.

Hand it off — then **keep it honest**

A spec is only as good as the prompt that delivers it. The order below front-loads the cheap step — **surfacing gaps before the agent fills them with guesses**. Every ambiguity it names is one you close before a wrong implementation buries itself three layers deep.

- **Zero-distance** (skill / declarative agent): building is deterministic, so the interviewer **emits the artifact** (SKILL.md / AGENT.md) — no build step.

- **Build-required** (coded feature/agent): hand off the build prompt; the building agent **plans** → **you approve** → **builds**.

■ The hand-off prompt BUILD-REQUIRED TARGETS — POINT THE AGENT AT /SPECS/[AGENT].MD, IN THIS ORDER

- 1 **Restate the outcome** in one sentence. Confirms it actually parsed the anchor.
- 2 **List ambiguities & missing info before starting**. The single highest-leverage line — gaps surfaced now are gaps you get to close.
- 3 Write the implementation **against the requirements section** — each one is already a test.
- 4 After building, **check each acceptance criterion one by one** — by running it, not asserting it.
- 5 **Do not mark complete** until every criterion passes.

Guardrail. Don't add anything outside **in-scope**. Don't pull packages outside **constraints** without flagging first.

Capture drift. Any architectural call the spec didn't cover gets appended to a **"decisions made"** block at the bottom of the file.

■ Two decision logs, not one • CLARIFIED — THEY ARE DIFFERENT SECTIONS WITH DIFFERENT AUTHORS

► Prior decisions — INPUT

You write these **before** the build. Settled context the agent must inherit. "Don't relitigate" — but it *may* flag one that looks unsafe or broken.

◄ Decisions made — OUTPUT

The agent appends these **during** the build. Next session reads them as Prior decisions, or the codebase drifts from its **source of truth**.

■ Make verification real • THE GAP AGENTS EXPLOIT — "DONE" IS A CLAIM, NOT A FACT

Traceability • ADD

Every requirement maps to **≥1 acceptance criterion**. An untraceable requirement is one nobody will verify.

Machine-checkable • ADD

A criterion is a **test or CI run**, not the agent's say-so. Agents over-report "complete" — make the check independent of the checker.

Cover the silent skips • ADD

Spell out the non-functional needs agents quietly drop:

security performance errors
observability ally privacy

■ What breaks, and when A SPEC WON'T FIX EVERYTHING — KNOW WHERE IT FAILS

! Too abstract

"SHALL provide a good UX" isn't a requirement. If a human can't test it, an agent can't verify it. **Every requirement needs a trigger and a response.**

! Criteria too loose

"Email sends correctly" fails; **"arrives within 5s of submission"** passes. Specificity is what makes a criterion verifiable.

! Whole project at once

One spec per agent, feature, or skill, a single meaningful unit. A spec for "the entire auth system" drifts and contradicts itself before the agent finishes reading.

! Never updated

Treat the spec like code: **version it, update it the moment things change**. Otherwise the next session starts from a diverged source of truth.

THE TEST

The **regeneration test**. Could a building agent rebuild this from the spec alone and produce **behaviourally identical** output? If yes, the spec is good. If no, you've just located exactly what's missing.

■ Put it to work DROP IT IN THE REPO • ONE SPEC PER AGENT, FEATURE, OR SKILL • RE-SPEC AS YOU GO

1 • Write

Save as `/specs/[agent].md` — six blocks, one agent, feature, or skill. Reuse the companion template.

2 • Hand off

Point the agent at the file with the 5-step prompt above. Let it list ambiguities first.

3 • Keep current

Agent appends `## decisions made`; you fold them back in & re-run. Version the spec like code.

Minimum viable spec when you're moving fast: Outcome + Scope (in/out) + 3 EARS requirements + their matching criteria. Everything else is upgrade, not entry fee.

■ SHALL/mandatory ■ WHEN/event ■ WHILE/state ■ IF/unwanted ■ WHERE/optional ■ • added this edition